Crowdstrike LAB: Unveiling Batch Script Obfuscation

An Innovative Approach to Deobfuscation through Syntax Parsing and Function Extension

Amr Wanli, Lukas Wenz

April 05, 2024

Rheinische Friedrich-Wilhelms-Universität Bonn
MA-INF 3229 - Lab Course: IT Security (semester course WT23/24)
Institute for Computer Science IV
Work group for IT-Security

UNIVERSITÄT BONN

Table of contents

1. Introduction

2. Methods

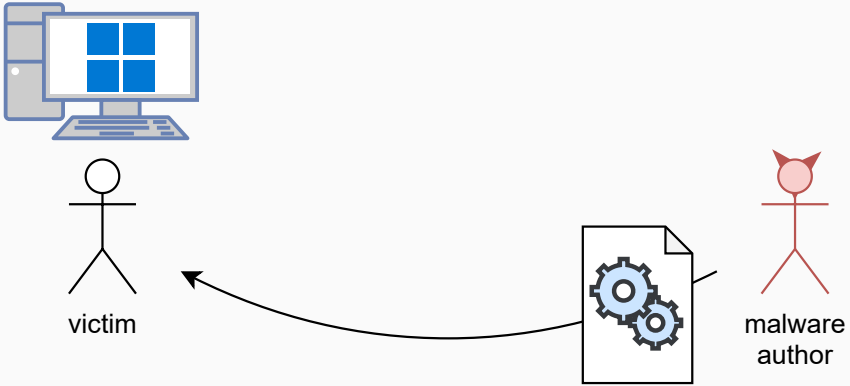
3. Frontend Evaluation

4. Backend Evaluation

5. Conclusion

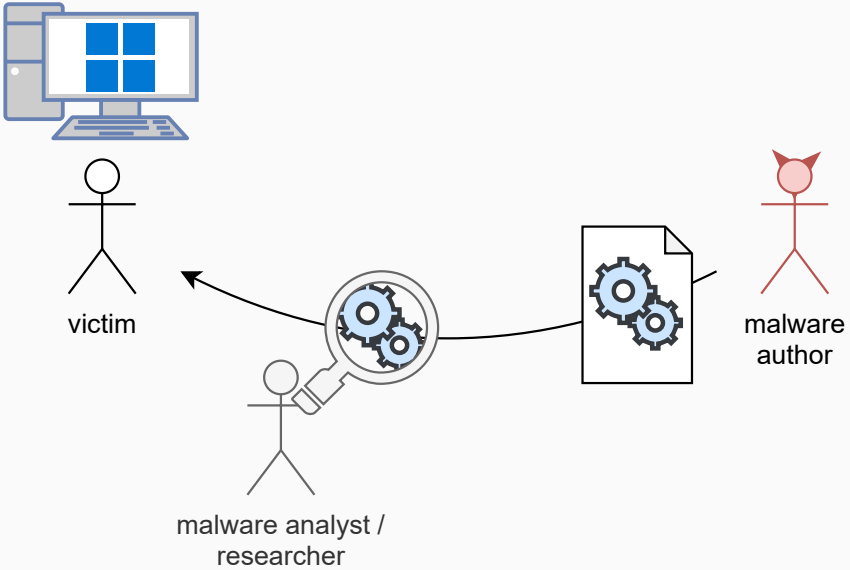
Intro

C:\introduction\context

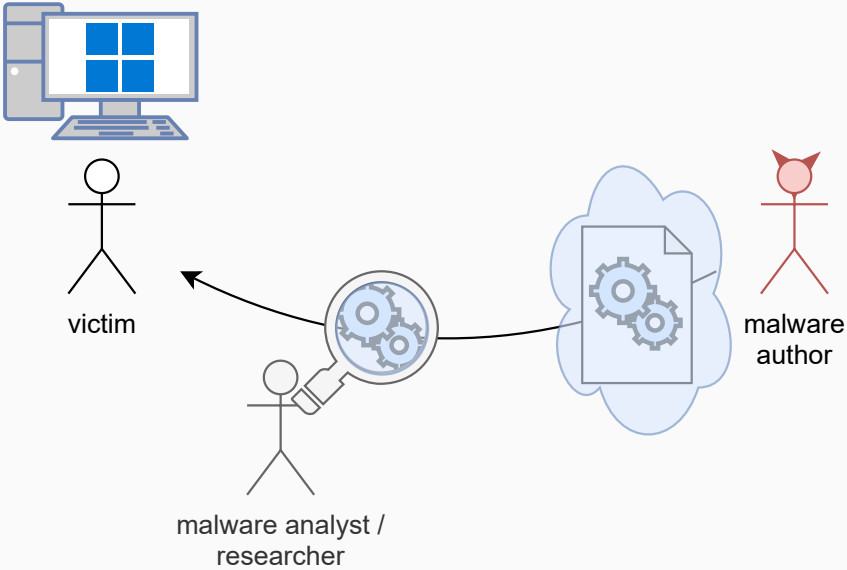


malware analyst / researcher

C:\introduction\context



C:\introduction\context



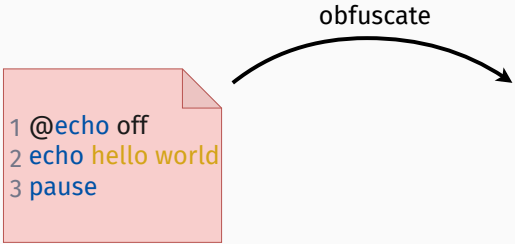


Figure 1: Typical batch (de)obfuscation example. Code partially taken from [4].

C:\introduction\what_is_(de)obfuscation

```
1 @echo off
2 echo hello world
3 pause
```

obfuscate

```
1 ;@;@@(@ch^c%法凡文無被^法%p%貼^是製是字護%.c%凡已人訊^這上%om 43^7)>n^ul&@e%魔法上
保神^人%%保凡護上人被^%%人神文法^被上%^cho o%字的保製^保這%f^%凡行人行製^法%f&c%這凡
護無^文法%%魔神^字製息護%^ls&&%ﻫﻫﻫ^%e%ﻫﻫﻫ%t
il|%%^%c%t%ﻫﻫﻫ%^%c%lii^%%^%ﻫﻫﻫ%iii=%0&^e%t%ﻫﻫﻫ%^%c
2 @:ABOBUS-OBFUSCATOR
3
4 ;SE^T "__author__=EscaLag"
5 ;SE^T "__github__=github.com/EscaLag/Abobus-obfuscator"
6
7 ;;@s^%魔這被此^訊上%e%無^訊無製人字%t /a +=1 >n^uL &F^o%此製魔訊^上已%%的法^訊字人字%r
%%a in (s%%^%%%
```

C:\introduction\what_is_(de)obfuscation

```
1 @echo off
2 echo hello world
3 pause
```

obfuscate →

```
1 ;@;@@(@ch^c%法凡文無被^法%p%貼^是製是字護%.c%凡已人訊^這上%om 43^7)>n^uI&@e%魔法上
  保神^人%%保凡護上人被^%%人神文法^被上%^cho o%字的保製^保這%f^%凡行人行製^法%f&c%這凡
  護無^文法%%魔神^字製息護%^Is&&s%ﺟﺴﻮ^~%e^%ﺟﺴﻮ^~%f
  iIl%%ﺟﺴﻮ^~%IIii^%ﺟﺴﻮ^~%iii=%0&s^e%ﺟﺴﻮ^~%t ﺟﺴﻮ^~%t ﺟﺴﻮ^~%t
```

C:\introduction\deobfuscators

- Deobfuscators are tools designed to reverse the process of obfuscation.
- Analyzed 2 popular deobfuscators on GitHub

C:\introduction\deobfuscators_(1)

```python
for l in lines:
    # ...

    if l.startswith("set ") and "=" in l:
        _, _, kv = l.partition(" ")
        key = kv.split("=")[0]
        value = kv.split("=")[1]
        print(f"[*] Found key: {key}, value: {value}")

    elif "%" in li and "~" in li:
        for key, value in env.items():
            # find and replace key with value
            # ...
```

The main part of a simple deobfuscator, taken from [6].

C:\introduction\deobfuscators_(2)

```
def get_value(self, variable):
    # This is where instances of a variable are replaced
    # by their value
    str_substitution = (
      r"([%!])(?P<variable>[\"^|!\w#$'()*+,-.?@\[\]`{}~\s+]+)"
      # ...
      r")?(\1)"
    )

# ...

if command == "set":
    # Interpreting `set` command
    var_name, var_value = self.interpret_set(normalized_comm[3:])
```

a more sophisticated deobfuscator that heavily relies on regexes [1]

C:\introduction\issues_with_naive_deobfuscators

C:\introduction\issues_with_naive_deobfuscators

1. no real understanding of the underlying grammar

C:\introduction\deobfuscator_(1)

the variable extractor from [6] processes texts naively without understanding of the underlying grammar:

```
if l.startswith("set ") and "=" in l:
    _, _, kv = l.partition(" ")
    key = kv.split("=")[0]
    value = kv.split("=")[1]
```

works well with

```
set VAR=VALUE          :: {"VAR": "VALUE"}
```

but what about?

```
set , ; VAR=VALUE      :: {", ; VAR": "VALUE"}
@ set , ; VAR=VALUE    :: {}
```

C:\introduction\issues_with_naive_deobfuscators

1. no real understanding of the underlying grammar
2. not pleasant to extend or to debug

C:\introduction\deobfuscator_(2)

Deobfuscator [1] heavily utilizes regexes to process batch syntax:

```
for_statement = (
    r"(?P<loop>(?P<for_statement>for)\s+"
    r"(?P<parameter>.+)"
    r"\s+IN\s+\((?P<in_set>[^\)]+)\)"
    r"\s+DO\s+"
    r"(?P<open_paren>\()?)(?P<command>[^\)]*)(?P<close_paren>\))?"
)
```

matches

```
for %%i in (1,2,3) do (
    echo %%i
    :: do something with it
)
```

C:\introduction\deobfuscator_(2)

```
str_substitution = (
    r"([%!])(?P<variable>[\"^|!\w#$'()*+,-.?@\[\]`{}~\s+]+)"
    r("
    r"(:~\s*(?P<idx>[+-]?\d+)\s*(?:,\s*(?P<len>[+-]?\d+))?\s*)|"
    r"(:(?P<s1>[^=]+)=(?P<s2>[^=]*))"
    r")?(\1)"
)
```

matches

```
%VAR:~1,-1%
```

C:\introduction\issues_with_naive_deobfuscators

1. no real understanding of the underlying grammar
2. not pleasant to extend or to debug
3. can be easily fooled

C:\introduction\issues_with_naive_deobfuscators

A trivial example of where popular deobfuscators fail to expand variable values correctly:

```
:: variable is initialized
set VAR="value1"

if 1 EQ 2 (
    :: variable is never updated, but naive deobfuscators will
    :: update value of the variable
    set VAR="value2"
)

:: the expanded value below is wrong, which defeats the
:: purpose of the deobfuscator!
echo the value is %VAR%
```

C:\introduction\issues_with_naive_deobfuscators

```
1  :: variable is initialized
2  set VAR="value1"
3
4  goto :END
5  set VAR="value2"
6
7  :END
8  :: wrong value in dictionary/environment!
9  echo the value is %VAR%
```

C:\introduction\deobfuscator - our approach

C:\introduction\deobfuscator - our approach

1. no real understanding of the underlying grammar?
   - ☐ Parse Batch!

C:\introduction\deobfuscator - our approach

1. no real understanding of the underlying grammar?
   - ☐ Parse Batch!
2. not pleasant to extend or to debug?

C:\introduction\deobfuscator - our approach

1. no real understanding of the underlying grammar?
   - ☐ **Parse Batch!**
2. not pleasant to extend or to debug?
   - ☐ **Need an extensible & developer-friendly framework!**

C:\introduction\deobfuscator - our approach

1. no real understanding of the underlying grammar?
   - ☐ **Parse Batch!**
2. not pleasant to extend or to debug?
   - ☐ **Need an extensible & developer-friendly framework!**
3. can be easily mitigated?

C:\introduction\deobfuscator - our approach

1. no real understanding of the underlying grammar?
   - ☐ **Parse Batch!**
2. not pleasant to extend or to debug?
   - ☐ **Need an extensible & developer-friendly framework!**
3. can be easily mitigated?
   - ☐ **Framework must be robust enough to understand:**
     - scopes
     - control flow
     - data flow
     - runtime environment
     - ...

Methods

C:\methods

1. no real understanding of the underlying grammar?
   - ☐ **Parse Batch!**
2. not pleasant to extend or to debug?
   - ☐ **Need an extensible & developer-friendly framework!**
3. can be easily mitigated?
   - ☐ **Framework must be robust**

C:\methods\choice_of_parser_generator_tool

C:\methods\choice_of_parser_generator_tool

1. active maintenance
2. documentation availability
3. Python integration options

erikrose / parsimonious

Figure 2: Logos taken from [10, 7].

Figure 3: Logo taken from [9].

C:\methods\choice_of_parser_generator_tool

Figure 2: Logos taken from [10, 7].

PARSIMONIOUS

1. active maintenance
2. documentation availability
3. Python integration options

erikrose / parsimonious

Figure 3: Logo taken from [9].

C:\methods\choice_of_parser_generator_tool

Figure 2: Logos taken from [10, 7].

GNU Bison

1. active maintenance
2. documentation availability
3. Python integration options

erikrose / parsimonious

Figure 3: Logo taken from [9].

C:\methods\choice_of_parser_generator_tool

Figure 2: Logos taken from [10, 7].

ANTLR

1. active maintenance
2. documentation availability
3. Python integration options

VS.

TREE-SITTER

1. active maintenance
2. documentation availability
3. Python integration options

erikrose / parsimonious

Figure 3: Logo taken from [9].

C:\methods\choice_of_parser_generator_tool

Figure 2: Logos taken from [10, 7].

ANTLR

1. active maintenance
2. documentation availability
3. Python integration options

VS.

TREE-SITTER

1. active maintenance
2. documentation availability
3. Python integration options

erikrose / parsimonious

Figure 3: Logo taken from [9].

C:\methods

1. no real understanding of the underlying grammar?
   - ☑ **Parse Batch!**
2. not pleasant to extend or to debug?
   - ☐ **Need an extensible & developer-friendly framework!**
3. can be easily mitigated?
   - ☐ **Framework must be robust**

C:\methods\architecture_overview

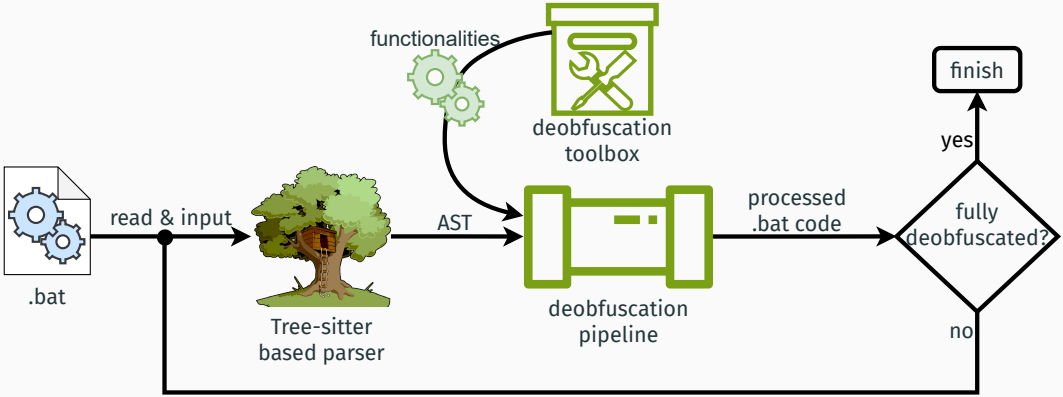


Figure 4: Framework architecture and functionality. Logo taken from [9].

C:\methods\architecture_overview

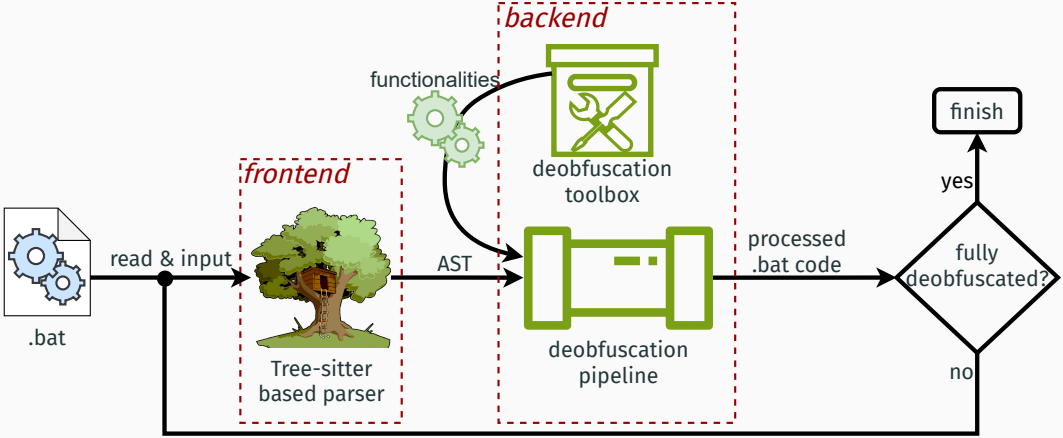


Figure 4: Framework architecture and functionality. Logo taken from [9].

C:\methods

1. no real understanding of the underlying grammar?
   - ☑ **Parse Batch!**
2. not pleasant to extend or to debug?
   - ☑ **Need an extensible & developer-friendly framework!**
3. can be easily mitigated?
   - ☐ **Framework must be robust**

Frontend Evaluation

C:\frontend_evaluation\corpus - evaluation methodology

- testing functionality of Tree-sitter
- corpus: formulation of test cases
- principle: comparison of expected and actual ASTs
- test-driven development

```
==========================
echo 1$
==========================

echo "Hello, world!"

---

(source_file
    (command
        (command_name)
        (text)))
```

C:\frontend_evaluation\corpus - test case combination example

```
1  set FILE="file.txt"
```

```
1  if exist %FILE% (
2    echo "File exists."
3  ) else (
4    echo "File does not exist."
5  )
```

```
(source_file
  (assignment
    (variable)
    (operator)
    (value))


  (if_statement
    (condition_statement
      (unary_identifier_expression
        (variable_substitution)))
    (command
      (command_name)
      (text))
    (else_clause
      (command
        (command_name)
        (text)))))
```

C:\frontend_evaluation\corpus - results

- all 144 test cases succeeded

```
commands:
  ✓ command escaped 1$
  ✓ command escaped 2$
  ✓ ~ command separator 1$
  ✓ ~ command separator 2$
  ✓ command separator 3$
  ✓ variable as command 1$
  ✓ variable as command 2$
  ✓ variable as command 3$
  ✓ variable as command 4$
  ✓ variable as command 5$
  ✓ variable as command 6$
  ✓ variable as command 7$
  ✓ variable in command 1$
  ✓ variable in command 2$
  ✓ variable in command 3$
  ✓ string as command 1$
  ✓ string as command 2$
  ✓ string as command 3$
  ✓ demo 1$
comments:
  ✓ comment 1
  ✓ comment 2
  ✓ comment 3
  ✓ comment 4
  ✓ comment 5
  ✓ comment 6
echo:
  ✓ echo 1$
  ✓ echo word 1$
  ✓ echo word 2$
  ✓ echo word 3$
  ✓ echo word 4$
  ✓ echo word 5$
  ✓ echo word 6$
  ✓ echo word 7$
  ✓ echo word 8$
  ✓ echo word 9$
  ✓ echo word 10$
  ✓ echo word 11$
  ✓ echo string 1$
  ✓ echo string 2$
  ✓ echo string 3$
  ✓ echo string 4$
  ✓ echo string 5$
  ✓ echo string 6$
  ✓ echo string 7$
  ✓ echo string 8$
  ✓ echo variable 1$
  ✓ ~ echo variable 2$
  ✓ echo variable 3$
  ✓ echo variable 4$
  ✓ echo variable 5$
  ✓ echo variable 6$
  ✓ echo variable 7$
  ✓ echo variable 8$
  ✓ echo variable 9$
  ✓ echo variable 10$
  ✓ echo variable 11$
  ✓ echo variable 12$
  ✓ echo variable 13$
  ✓ echo delimiter 1$
  ✓ echo delimiter 2$
  ✓ echo delimiter 3$
  ✓ echo delimiter 4$
  ✓ echo delimiter 5$
  ✓ echo concat 1$
  ✓ echo concat 2$
  ✓ echo concat 3$
  ✓ echo flag 1$
  ✓ ~ echo flag 2$
  ✓ ~ echo flag 3$
  ✓ ~ echo flag 4$
  ✓ ~ echo flag 5$
  ✓ echo flag 6$
  ✓ echo flag 7$
  ✓ echo flag 8$
  ✓ echo flag 9$
  ✓ echo flag 10$
  ✓ ~ echo flag 11$
  ✓ echo flag 12$
  ✓ ~ echo flag 13$
  ✓ echo flag 14$
  ✓ ~ echo flag 15$
  ✓ echo flag 16$
  ✓ echo complex 1$
  ✓ echo complex 2$
if:
  ✓ if statements - variations
  ✓ if statements - comparison and %~
  ✓ if statements - unary_numerical_expression
  ✓ if else statements - multiline variation
  ✓ if else statements - if parentheses only
  ✓ if else statements - both parentheses
  ✓ if else statements - hyperbolic separator usage
  ✓ if else statements - some more escaping
  ✓ if else statements - no separator in front of else
  ✓ if else statements - if command and closing if parenthesis on different lines
for:
  ✓ for loop counting up
  ✓ for statements - one liners
  ✓ for statements - l-switch empty bracket
  ✓ for statements - options w/ backquotes
  ✓ for statements - some escaping
statements:
  ✓ Echo and comments
  ✓ Echo multiple words
  ✓ Terminators - Testing statements
  ✓ Switches and attributes
  ✓ Basic conditional execution
  ✓ Simple pipelines
  ✓ Double pipeline & rem-String
echo:
  ✓ echo complex 3$
  ✓ echo complex 4$
  ✓ echo complex 5$
labels:
  ✓ label 1$
  ✓ label 2$
  ✓ label 3$
  ✓ set 10$
  ✓ set 11$
  ✓ set 12$
  ✓ set 13$
  ✓ set 14$
  ✓ set 15$
  ✓ set 16$
  ✓ set 17$
  ✓ set 18$
  ✓ set 19$
```

C:\frontend_evaluation\empirical_analysis - evaluation methodology

- parsing **382 batch files**, sourced from publicly available real-world contexts
- compiling all expected ASTs is intractible
- instead: error analysis
- **core ideas**:
  1. small number of errors $\Rightarrow$ good coverage $\Rightarrow$ parser empirically covers syntax well
  2. assumption: reported errors are valid
  3. Tree-sitter errors are managable

C:\frontend_evaluation\empirical_analysis - results

- parsing **382 batch files**, sourced from publicly available real-world contexts
- success rate of approx. **63.35%**

#test cases	thereof successful	thereof erroneous		
		in sum	thereof sp. keyword	thereof redir.
382	241	140	83	29

sp. keyword = special keyword of Tree-sitter's reported errors (*MISSING*, *UNEXPECTED*)

redir = batch command redirection

Backend Evaluation

C:\backend_evaluation

- assess effectiveness against various batch obfuscators
- evaluate ease of integrating new deobfuscation techniques into framework
- identify any potential challenges or limitations

C:\backend_evaluation

- 288 test scripts
- scripts vary in size & complexity:
  - an empty file, simple & complex batch scripts
- 3 obfuscators:
  - Baum1810 [2]
  - Abobus [4]
  - SomalifuscatorV2 [5]
- total of 864 tests
- 2 deobfuscation techniques:
  - variable expansion
  - command name cleaners

C:\backend_evaluation

Obfuscator	GOOD	OK	UF	UB	BAD	FAIL
Baum1810	38%	18%	26%	6%	12%	0
Abobus	<1%	0	0	31%	0	68%
SomalifuscatorV2	0	0	0	4%	0	96%

C:\backend_evaluation\readability_of_deobfuscated_code

- deobfuscating Baum1810 yielded correctly deobfuscated scripts
- deobfuscating Abobus and SomalifuscatorV2 decrease readability:
  - spaghetti code
  - dead code
  - opaque predicates
  - single-iteration for loops

DEMO

Conclusion

C:\conclusion

1. no real understanding of the underlying grammar?
   - ☑ **Parse Batch!**
2. not pleasant to extend or to debug?
   - ☑ **Need an extensible & developer-friendly framework!**
3. can be easily mitigated?
   - ☐ **Framework must be robust**

Thank you for listening.

Questions?

References i

Aaron Meese.
Batch Deobfuscator, 2024.
URL: *https://github.com/TargetPackage/batch_deobfuscator* (visited on 03/16/2024).

Baum1810.
Batch Obfuscator, 2024.
URL: *https://github.com/baum1810/batchobfuscator* (visited on 03/16/2024).

Chris Williams.
Unofficial JavaScript logo by Chris Williams, 2024.
URL: *https://commons.wikimedia.org/wiki/File:Unofficial_JavaScript_logo_2* (visited on 03/24/2024).

References ii

EscaLag.
Abobus obfuscator, 2024.
URL: *https://github.com/EscaLag/Abobus-obfuscator* (visited on 03/16/2024).

KDot227.
SomalifuscatorV2, 2024.
URL: *https://github.com/KDot227/SomalifuscatorV2* (visited on 03/16/2024).

Manas Bellani.
Bat File Deobfuscate, 2024.
URL: *https://github.com/manasmbellani/mycode_public* (visited on 03/16/2024).

References iii

Terence Parr.
ANTLR v4 Logo, 2024.
URL: *https://plugins.jetbrains.com/plugin/7358-antlr-v4* (visited on 03/25/2024).

The Python Software Foundation.
The Python Logo, 2024.
URL: *https://www.python.org/community/logos/* (visited on 03/24/2024).

Tree-sitter.
Tree-sitter Introduction, 2024.
URL: *https://tree-sitter.github.io/* (visited on 03/16/2024).

Victor Siame.
A GNU Head - Logo, 2006.
URL: *https://www.gnu.org/graphics/graphics.html* (visited on 03/25/2024).

BACKUP SLIDES

C:\design_goal_discussion

1. no real understanding of the underlying grammar?
   - ☑ **Parse Batch!**
2. not pleasant to extend or to debug?
   - ☑ **Need an extensible & developer-friendly framework!**
3. can be easily mitigated?
   - ☐ **Framework must be robust**

design goals:

- maintainability
- extensibility
- usability

C:\design_goal_discussion\maintainability

programming languages

libraries

other tools

C:\design_goal_discussion\maintainability

programming languages

easy to maintain programming languages:

- pure python backend
- minimal JavaScript knowledge required for the frontend

libraries

other tools

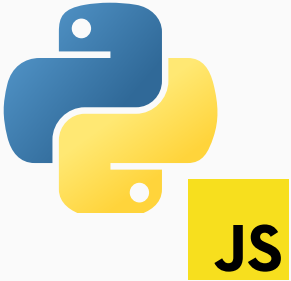


Figure 5: Logos taken from [8, 3].

C:\design_goal_discussion\maintainability

programming languages

easy to maintain programming languages:

- pure python backend
- minimal JavaScript knowledge required for the frontend

libraries

lightweight python dependencies

- backend depends only on `os`, `re` and `logging`

other tools

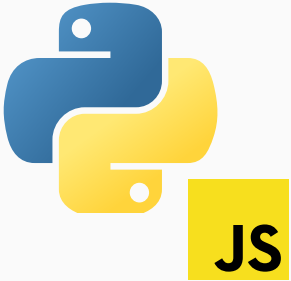


Figure 5: Logos taken from [8, 3].

C:\design_goal_discussion\maintainability

programming languages

easy to maintain programming languages:

- pure python backend
- minimal JavaScript knowledge required for the frontend

libraries

lightweight python dependencies

- backend depends only on `os`, `re` and `logging`

other tools

Tree-sitter is well maintained itself

- regularily updated, both library and bindings

Figure 5: Logos taken from [8, 3, 9].

C:\design_goal_discussion\extensibility

programming languages

backbone tool

limitations

C:\design_goal_discussion\extensibility

programming languages

easy to extend programming language:

- pure python backend → extensive library support

backbone tool

limitations

Figure 6: Logos taken from [8].

C:\design_goal_discussion\extensibility

programming languages

easy to extend programming language:

- pure python backend → extensive library support

backbone tool

Tree-sitter comes with good Python bindings

- established tool with widespread community contributions

limitations

Figure 6: Logos taken from [8, 9].

C:\design_goal_discussion\extensibility

programming languages

easy to extend programming language:

- pure python backend → extensive library support

backbone tool

Tree-sitter comes with good Python bindings

- established tool with widespread community contributions

limitations

Tree-sitter generates immutable ASTs

- challenging for modifications
- workaround available

Figure 6: Logos taken from [8, 9].

C:\design_goal_discussion\usability

installation

ease of use

C:\design_goal_discussion\usability

installation

only Tree-sitter & Python required:

- has two dependencies itself: C compiler & Node.js
- publishing to Tree-sitter would allow easy importing

ease of use

C:\design_goal_discussion\usability

installation

only Tree-sitter & Python required:

- has two dependencies itself: C compiler & Node.js
- publishing to Tree-sitter would allow easy importing

ease of use

modular approach in the backend:

- processed pipeline
- mutability
- quality-of-life functions

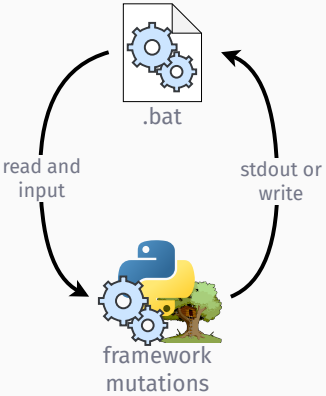


Figure 7: Basic mutation principle. Logos taken from [8, 9].